

```
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.preprocessing import LabelEncoder
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.tree import DecisionTreeClassifier
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import accuracy_score, confusion_matrix
```

```
df=pd.read_csv("customer_retail csv file.csv")
```

```
print(df.head())
```

	InvoiceNo	StockCode	Description	Quantity	\
0	536365	85123A	WHITE HANGING HEART T-LIGHT HOLDER	6.0	
1	536365	71053	WHITE METAL LANTERN	6.0	
2	536365	84406B	CREAM CUPID HEARTS COAT HANGER	8.0	
3	536365	84029G	KNITTED UNION FLAG HOT WATER BOTTLE	6.0	
4	536365	84029E	RED WOOLLY HOTTIE WHITE HEART.	6.0	

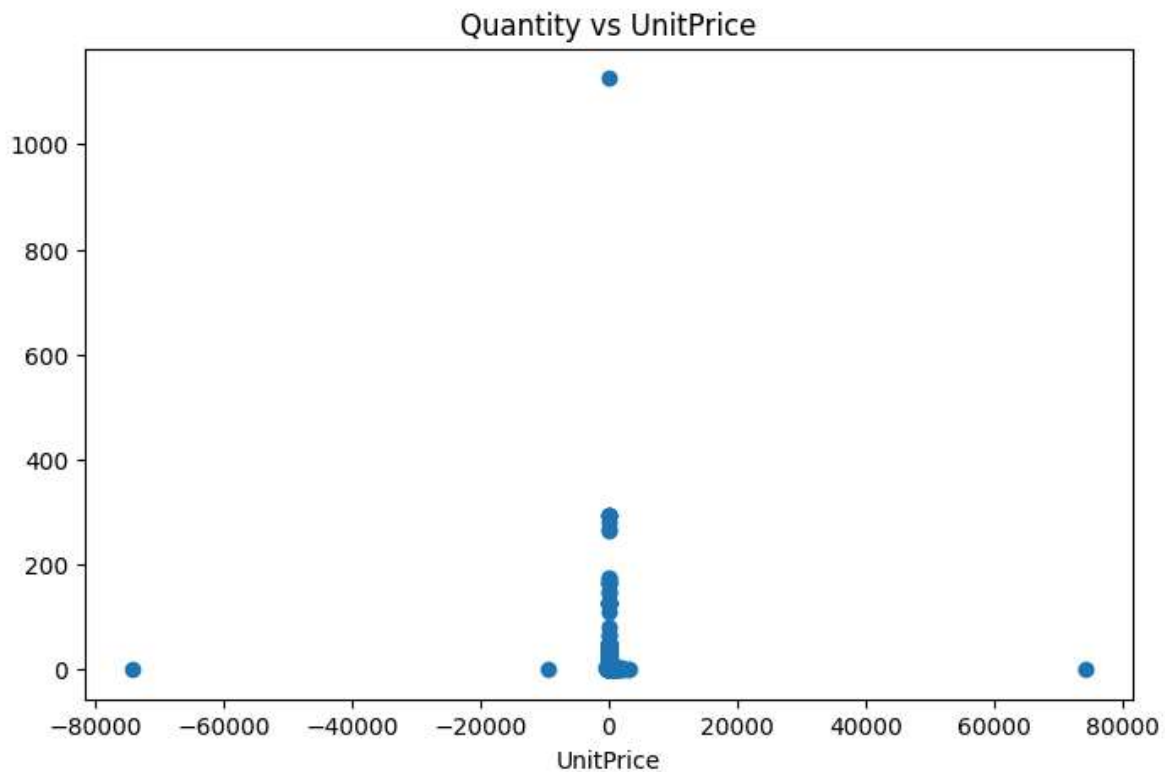
	InvoiceDate	UnitPrice	CustomerID	Country
0	01-12-2010 08:26	2.55	17850.0	United Kingdom
1	01-12-2010 08:26	3.39	17850.0	United Kingdom
2	01-12-2010 08:26	2.75	17850.0	United Kingdom
3	01-12-2010 08:26	3.39	17850.0	United Kingdom
4	01-12-2010 08:26	3.39	17850.0	United Kingdom

```
df=df.dropna()
```

```
df=df[['Quantity','UnitPrice','Country']]
```

```
encoder=LabelEncoder()
df['Country_encoded']=encoder.fit_transform(df['Country'])
```

```
plt.figure(figsize=(8,5))
plt.scatter(df['Quantity'],df['UnitPrice'])
plt.xlabel('Quantity')
plt.ylabel('UnitPrice')
plt.title('Quantity vs UnitPrice')
plt.show()
```



```
x=df[['Quantity','UnitPrice']]
y=df['Country_encoded']
x_train,x_test,y_train,y_test=train_test_split(x,y,test_size=0.2,random_state=42)
```

```
print("LogisticRegression")
log_model=LogisticRegression()
log_model.fit(x_train,y_train)
y_pred=log_model.predict(x_test)
accuracy=accuracy_score(y_test,y_pred)
print("Accuracy",accuracy)
cm=confusion_matrix(y_test,y_pred)
print("Confusion Matrix")
print(cm)
```

```
LogisticRegression
Accuracy 0.9024231370505471
Confusion Matrix
```

```
[ [ 0  0  0  0  0  0  0  0  0  0  0  0  0  0
    0  0  0  0  0  0  0  0 20]
  [ 0  0  0  0  0  0  0  0  0  0  0  0  0  0
    0  0  0  0  0  0  0  0  1]
  [ 0  0  0  0  0  0  0  0  0  0  0  0  0  0
    0  0  0  0  0  0  0  0 22]
  [ 0  0  0  0  0  0  0  0  0  0  0  0  0  0
    0  0  0  0  0  0  0  0  3]
  [ 0  0  0  0  0  0  0  0  0  0  0  0  0  0
    0  0  0  0  0  0  0  0 25]
  [ 0  0  0  0  0  0  0  0  0  0  0  0  0  0
    0  0  0  0  0  0  0  0  6]
  [ 0  0  0  0  0  0  0  0  0  0  0  0  0  0
    0  0  0  0  0  0  0  0 82]
  [ 0  0  0  0  0  0  0  0  0  0  0  0  0  0
    0  0  0  0  0  0  0  0  1]
  [ 0  0  0  0  0  0  0  0  0  0  0  0  0  0]
```

```

0 0 0 0 0 0 0 155]
[ 0 0 0 0 0 0 0 0 0 0 0 0 0 0
0 0 0 0 0 0 0 178]
[ 0 0 0 0 0 0 0 0 0 0 0 0 0 0
0 0 0 0 0 0 0 4]
[ 0 0 0 0 0 0 0 0 0 0 0 0 0 0
0 0 0 0 0 0 0 23]
[ 0 0 0 0 0 0 0 0 0 0 0 0 0 0
0 0 0 0 0 0 0 10]
[ 0 0 0 0 0 0 0 0 0 0 0 0 0 0
0 0 0 0 0 0 0 6]
[ 0 0 0 0 0 0 0 0 0 0 0 0 0 0
0 0 0 0 0 0 0 28]
[ 0 0 0 0 0 0 0 0 0 0 0 0 0 0
0 0 0 0 0 0 0 36]
[ 0 0 0 0 0 0 0 0 0 0 0 0 0 0
0 0 0 0 0 0 0 9]
[ 0 0 0 0 0 0 0 0 0 0 0 0 0 0
0 0 0 0 0 0 0 48]
[ 0 0 0 0 0 0 0 0 0 0 0 0 0 0
0 0 0 0 0 0 0 69]
[ 0 0 0 0 0 0 0 0 0 0 0 0 0 0
0 0 0 0 0 0 0 10]
[ 0 0 0 0 0 0 0 0 0 0 0 0 0 0
0 0 0 0 0 0 0 11]
[ 0 0 0 0 0 0 0 0 0 0 0 0 0 0
0 0 0 0 1 1 0 6927]]

```

/usr/local/lib/python3.12/dist-packages/sklearn/linear_model/_logistic.py:465: ConvergenceWarning:
STOP: TOTAL NO. OF ITERATIONS REACHED LIMIT.

Increase the number of iterations (max_iter) or scale the data as shown in:

<https://scikit-learn.org/stable/modules/preprocessing.html>

Please also refer to the documentation for alternative solver options:

https://scikit-learn.org/stable/modules/linear_model.html#logistic-regression

```
n_iter_i = _check_optimize_result(
```

```

print("Decision Tree")
dt_model=DecisionTreeClassifier()
dt_model.fit(x_train,y_train)
y_pred=dt_model.predict(x_test)
accuracy=accuracy_score(y_test,y_pred)
print("Accuracy",accuracy)
print(confusion_matrix(y_test,y_pred))

```

Decision Tree

Accuracy 0.9007295466388744

```

[[ 2 0 0 0 0 0 0 0 0 0 0 0 1 0
0 0 0 0 0 0 0 17]
[ 0 0 0 0 0 0 0 0 0 0 0 0 0 0
0 0 0 0 0 0 0 1]
[ 0 0 1 0 0 0 0 0 0 0 0 0 0 0
0 0 0 0 0 0 0 21]
[ 0 0 0 0 0 0 0 0 0 0 0 0 0 0
0 0 0 0 0 0 0 3]
[ 0 0 0 0 1 0 0 0 0 0 0 0 0 0
0 0 0 0 0 0 0 24]
[ 0 0 0 0 0 0 0 0 0 0 0 0 0 0
0 0 0 0 0 0 0 6]
[ 0 0 0 0 0 0 6 0 0 1 0 0 0 0
0 0 0 0 0 0 0 75]
[ 0 0 0 0 0 0 0 0 0 0 0 0 0 0
0 0 0 0 0 0 0 1]
[ 0 0 0 0 0 0 0 0 2 3 0 0 0 0
0 0 0 0 0 0 0 150]

```

```
[ 0 0 0 0 0 0 1 0 3 8 0 0 0 0
 0 0 0 0 0 0 0 166]
[ 0 0 0 0 0 0 0 0 0 0 0 0 0 0
 0 1 0 0 0 0 0 3]
[ 0 0 0 0 0 0 0 0 0 1 0 0 0 0
 0 0 0 0 0 0 0 22]
[ 0 0 0 0 0 0 0 0 0 0 0 0 0 0
 0 0 0 0 0 0 0 10]
[ 0 0 0 0 0 0 0 0 0 0 0 0 0 0
 0 0 0 0 0 0 0 6]
[ 0 0 0 0 0 0 1 0 0 0 0 0 1 0
 5 0 0 0 0 0 0 21]
[ 0 0 0 0 0 0 0 0 0 0 0 0 0 0
 0 0 0 0 0 0 0 36]
[ 0 0 0 0 0 0 0 0 0 0 0 0 0 0
 0 0 0 0 0 0 0 9]
[ 0 0 0 0 0 0 0 0 0 0 0 0 0 0
 0 0 0 1 1 0 0 46]
[ 0 0 0 0 0 0 0 0 0 0 0 0 0 0
 0 0 0 1 1 0 0 67]
[ 0 0 0 0 0 0 0 0 0 0 0 0 0 0
 0 0 0 0 0 1 0 9]
[ 0 0 0 0 0 0 0 0 0 0 0 0 0 0
 0 0 0 0 0 0 0 11]
[ 5 0 1 0 1 0 10 3 3 9 0 0 6 0
 3 2 0 0 0 0 0 6886]]
```

```
print("KNN")
knn_model=KNeighborsClassifier()
knn_model.fit(x_train,y_train)
y_pred=knn_model.predict(x_test)
accuracy=accuracy_score(y_test,y_pred)
print("Accuracy",accuracy)
print(confusion_matrix(y_test,y_pred))
```

```
KNN
Accuracy 0.9007295466388744
[[ 0 0 0 0 0 0 0 0 0 0 0 0 1 0
 0 0 0 0 0 1 0 18]
 [ 0 0 0 0 0 0 0 0 0 0 0 0 0 0
 0 0 0 0 0 0 0 1]
 [ 0 0 1 0 0 0 0 0 0 0 0 0 0 0
 0 0 0 0 0 0 0 21]
 [ 0 0 0 0 0 0 0 0 0 0 0 0 0 0
 0 0 0 0 0 0 0 3]
 [ 0 0 0 0 0 0 0 0 0 0 0 0 0 0
 0 0 0 0 0 0 0 25]
 [ 0 0 0 0 0 0 0 0 0 0 0 0 0 0
 0 0 0 0 0 0 0 6]
 [ 1 0 0 0 0 0 0 4 0 0 0 0 0 0
 0 0 0 0 0 0 0 77]
 [ 0 0 0 0 0 0 0 0 0 0 0 0 0 0
 0 0 0 0 0 0 0 1]
 [ 0 1 0 0 0 0 0 0 2 2 0 0 0 1
 0 0 0 0 0 0 0 149]
 [ 2 0 0 0 0 0 0 0 5 3 0 0 0 0
 0 0 0 0 0 0 0 168]
 [ 0 0 0 0 0 0 0 0 0 0 0 0 0 0
 0 0 0 0 0 0 0 4]
 [ 0 0 0 0 0 0 0 0 0 0 0 0 0 0
 0 0 0 0 0 0 0 23]
 [ 0 0 0 0 0 0 0 0 0 1 0 0 0 0
 1 0 0 0 0 0 0 8]
 [ 0 0 0 0 0 0 0 0 0 1 0 0 0 1
 0 0 0 0 0 0 0 4]]
```

```
[ 0  0  0  0  0  0  1  0  0  0  0  0  1  0
 4  0  0  0  0  0  0  0 22]
[ 0  0  0  0  0  0  0  0  0  3  0  0  0  0
 0  0  0  0  0  0  0  0 33]
[ 0  0  0  0  0  0  0  0  0  0  0  0  0  0
 0  0  0  0  0  0  0  0  9]
[ 0  0  0  0  0  0  0  0  0  0  0  0  0  0
 0  0  0  4  1  0  0  0 43]
[ 0  0  0  0  0  0  0  0  0  0  0  0  0  0
 0  0  0  1  0  0  0  0 68]
[ 0  0  0  0  0  0  0  0  0  0  0  0  0  0
 0  0  0  0  0  0  1  0  9]
[ 0  0  0  0  0  0  0  0  0  0  0  0  0  0
 0  0  0  0  0  0  0  0 11]
[ 0  0  0  0  0  0  4  1  2  5  0  0  5  5
 4  6  0  0  0  3  0 6894]]
```

```
models=['Logistic Regression','Decision tree','KNN']
accuracies=[accuracy,accuracy,accuracy]
```

```
plt.figure(figsize=(8,5))
plt.bar(models,accuracies)
plt.xlabel('Models')
plt.ylabel('Accuracy')
plt.title('Model Accuracy Comparison')
plt.show()
```

